

Artificial Intelligence

人工智能实验

机器学习基础

中山大学计算机学院
2026年春季

目录

1. 理论课内容回顾

1.1 人工神经网络介绍

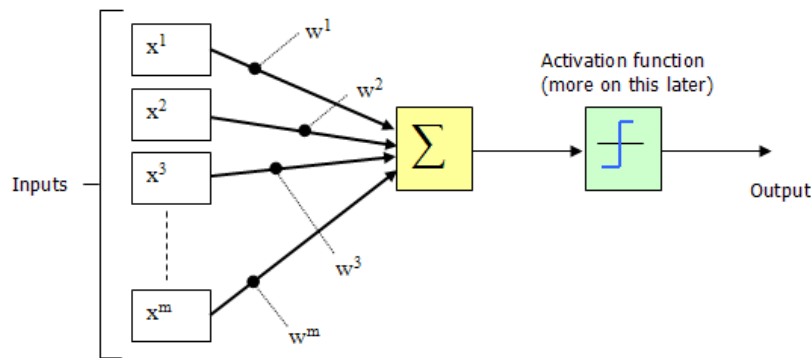
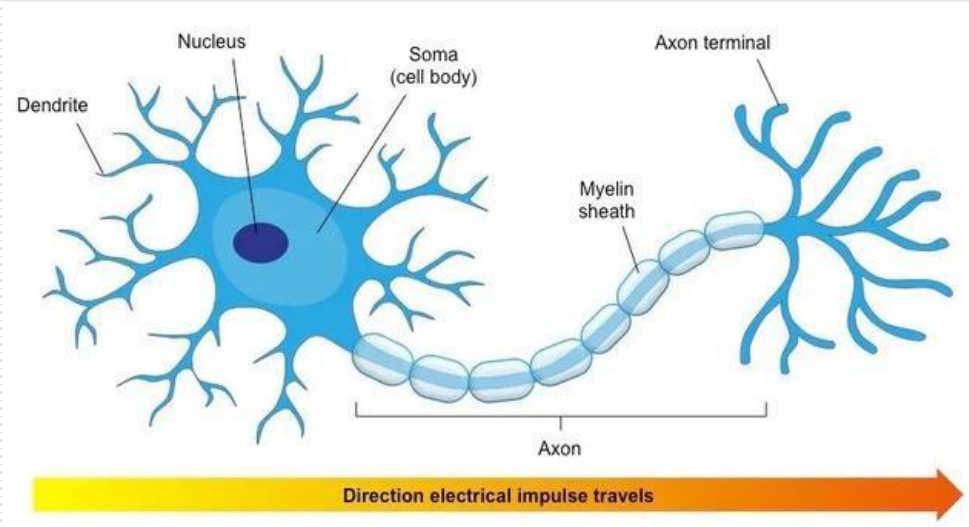
2. 实验任务

2.1 购房预测分类任务

3. 作业提交说明

1.1 人工神经网络介绍

- 神经网络采用了仿生学的思想，通过模拟生物神经网络的结构和功能来实现建模

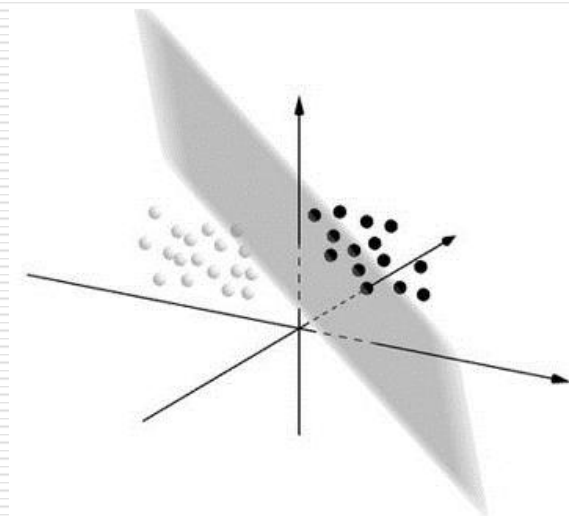
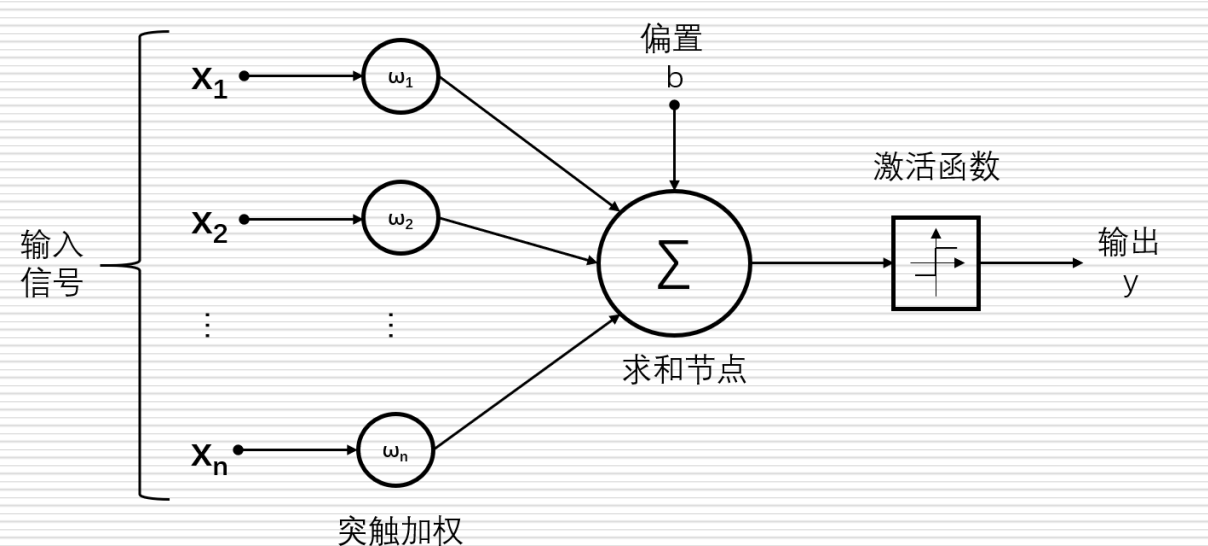


$$y = f(\sum_{i=1}^n w_i x_i - \theta)$$

1.1 人工神经网络介绍

□ 单层感知机

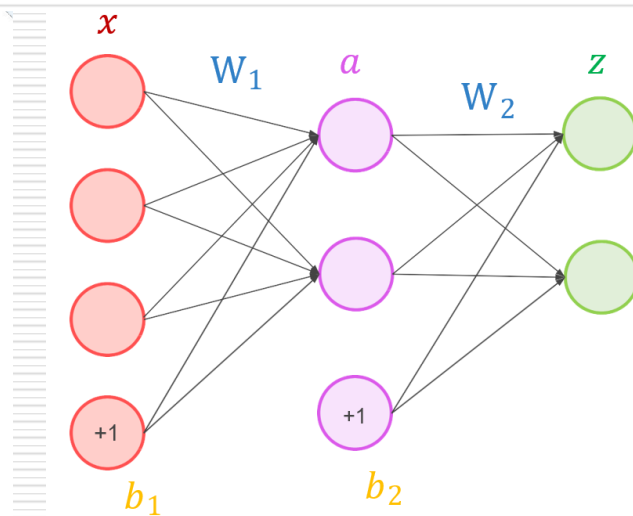
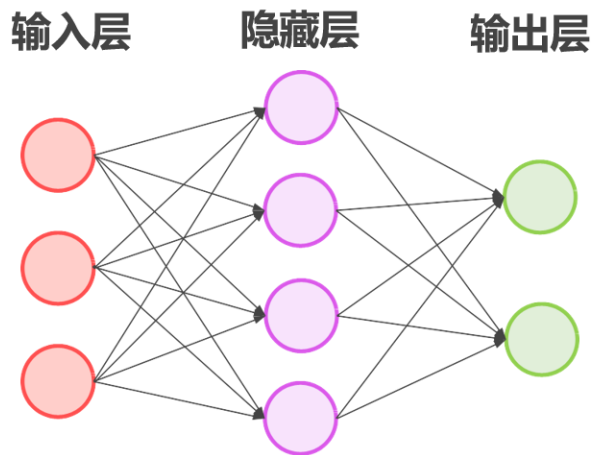
- 由于M-P神经元模型参数需要事先设定好，为了能够自适应学习出所需要的参数，研究人员就提出了单层感知机(Perceptron)
- 感知机的基本公式为： $y(x)=\text{sign}(wx+b)$
- sign 为符号函数，当自变量为正数时取值为1，否则取值为0



1.1 人工神经网络介绍

□ 多层感知机（Multi-Layer Perceptron，简称MLP）

- 包含三个层次：一个输入层，一个或多个中间层 (也叫隐藏层, hidden layer) 和一个输出层
- 输入层与输出层的节点数是固定的，中间层则可以自由指定
- MLP通常还会引入偏置单元**偏置单元** b



$$a = g_1(W_1 \times x + b_1)$$

$$\begin{aligned} z &= g_2(W_2 \times a + b_2) \\ &= g_2(W_2 \times g_1(W_1 \times x + b_1) + b_2) \end{aligned}$$

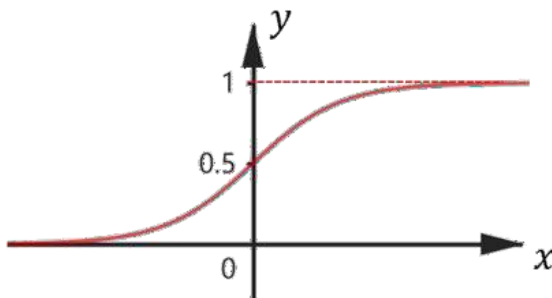
1.1 人工神经网络介绍

□ 激活函数

■ 常用激活函数：

□ sigmoid、Relu、tanh

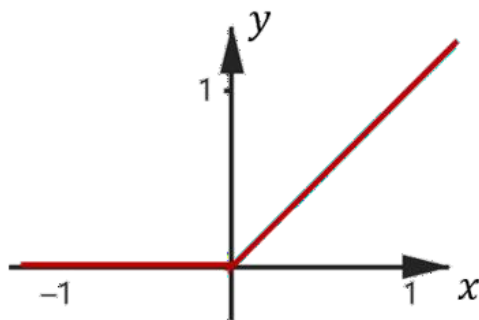
$$\text{sigmoid}(x) = \frac{1}{1 + e^{-x}}$$



(a) sigmoid

多用于多层神经网络(MLP)

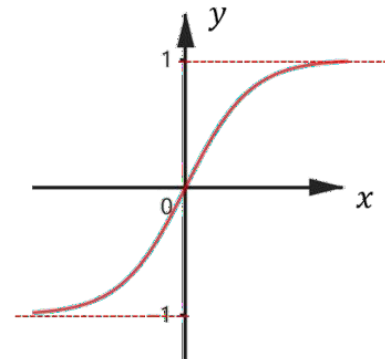
$$\text{ReLU}(x) = \max(0, x)$$



(b) ReLU

多用于深层神经网络(DNN)

$$\text{tanh}(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$



(c) tanh

多用于深层神经网络(DNN)

1.1 人工神经网络介绍

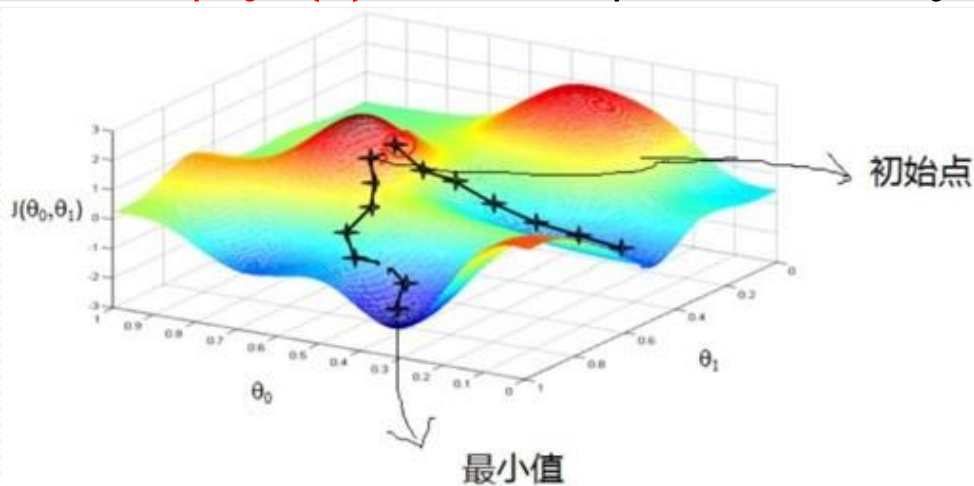
□ 损失函数

- 作用：为了衡量网络表现是否良好，并为之后的网络参数优化提供指导。
- 常见的用在分类任务上的损失函数：
 - 均方误差(MSE): $L_{MSE} = \frac{1}{2n} \sum_{i=1}^n (y^{(i)} - \hat{y}^{(i)})^2$
 - 交叉熵: $L_{CE} = - \sum_i^n y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})$

1.1 人工神经网络介绍

□ 梯度下降

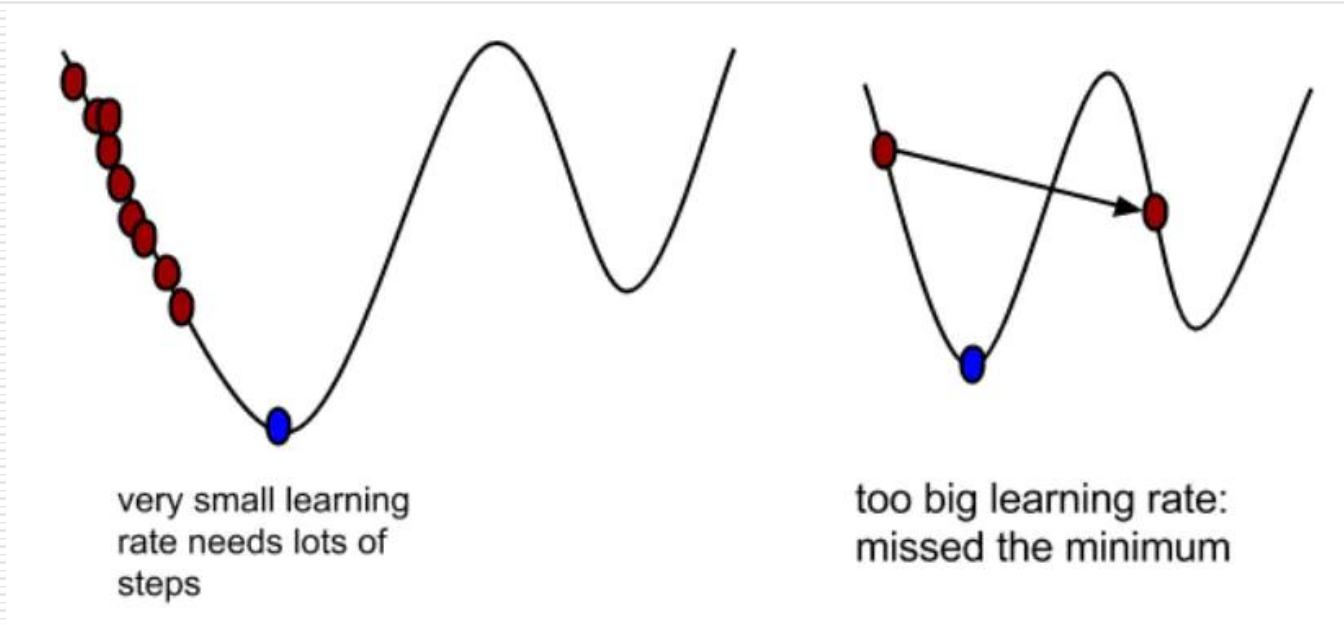
- 梯度定义：梯度是一个向量，表示某一函数在该点出的方向导数沿着该方向取得最大值。
- 也就是说该点处沿着梯度的方向变化最快，变化率最大
 - 沿着梯度方向容易找到函数最大值
 - 沿着梯度方向的反方向，容易找到函数最小值
- 梯度下降的一般公式为： $\theta = \theta - \eta \nabla_{\theta} L(\theta)$ ，其中， η 是学习率， ∇_{θ} 是对 θ 的梯度， θ 是参数



1.1 人工神经网络介绍

□ 学习率

- 学习率限制了下一步能到达的地方
- 如果学习率太小，可能很难达到最小值
- 如果学习率太大，则会错过最小值，无法收敛



2.1 购房预测分类任务

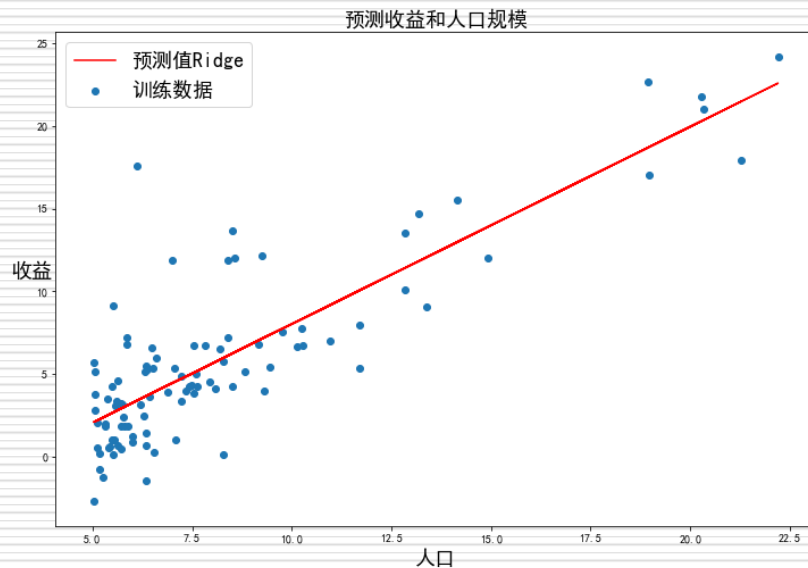
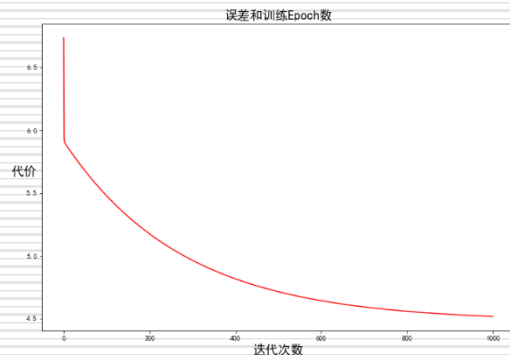
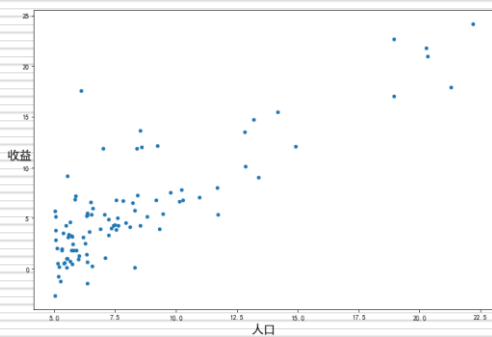
□ 购房预测分类任务

■ 利用**感知机算法**在给定数据集完成购房预测训练。

■ 要求：

□ 选择合适的损失函数，利用训练集完成网络训练，画出数据可视化图、loss曲线图。

□ 单层感知机示例：



2.1 购房预测分类任务

□ 购房预测分类任务

- 利用**感知机算法**在给定数据集完成购房预测训练

<https://www.kaggle.com/c/house-prices-advanced-regression-techniques>

1. 假定网络为多层感知机，网络输出为 $\hat{y} = MLP(X_{train})$ ， X 为房子的特征， MLP 为多层神经网络， W 为MLP的参数， $\hat{y}$ 为预测的房价
2. 设置损失函数为 L_{MSE} ，并随机初始化网络参数
3. 当满足终止条件时，终止优化，否则继续
4. 计算网络输出 $\hat{y} = MLP(X_{train})$ ，以及损失 $L_{MSE} = (MLP(X_{train}) - Y)^2$ ， Y 为真实房价
5. 对所有网络参数求导 $\frac{\partial L_{MSE}}{\partial W}$
6. 根据 $W = W - \eta \frac{\partial L_{MSE}}{\partial W}$ 更新参数 W ， η 为学习率（步长）
7. 跳转到3